01

```
PS C:\Users\MX971B> # Atskaites ID: RVT-4417
PS C:\Users\MX971B>
PS C:\Users\MX971B> $os = Get-CimInstance -ClassName Win32_OperatingSystem
PS C:\Users\MX971B> $processi = Get-Process
PS C:\Users\MX971B>
PS C:\Users\MX971B> Write-Output "Datora nosaukums: $env:COMPUTERNAME"
Datora nosaukums: DESKTOP-8JFL60V
PS C:\Users\MX971B> Write-Output "Operētājsistēma: $($os.Caption)"
Operētājsistēma: Microsoft Windows 11 Pro
PS C:\Users\MX971B> Write-Output "Būvējuma numurs: $($os.BuildNumber)"
Būvējuma numurs: 26200
PS C:\Users\MX971B> Write-Output "PowerShell versija: $($PSVersionTable.PSVersion)"
PowerShell versija: 5.1.26100.9711
PS C:\Users\MX971B> Write-Output "Datums un laiks: $(Get-Date)"
Datums un laiks: 10/18/2026 23:15:24
PS C:\Users\MX971B> Write-Output "Darbojošos procesu skaits: $($processi.Count)"
Darbojošos procesu skaits: 321
PS C:\Users\MX971B>
```

02

```
PS C:\Users\MX971B> # Atskaites ID: RVT-4417
PS C:\Users\MX971B>
PS C:\Users\MX971B> $processi = Get-Process |
>>     Sort-Object -Property WorkingSet64 -Descending |
>>     Select-Object -First 10 |
>>     Sort-Object -Property WorkingSet64
PS C:\Users\MX971B>
PS C:\Users\MX971B> foreach ($process in $processi) {
>>     $atmina = [math]::Round($process.WorkingSet64 / 1MB, 1)
>>     Write-Output "Nosaukums: $($process.ProcessName); ID: $($process.Id); Atmiņa: $atmina MB"
>> }
Nosaukums: msedge; ID: 27592; Atmiņa: 301 MB
Nosaukums: msedge; ID: 27300; Atmiņa: 320.2 MB
Nosaukums: msedge; ID: 6920; Atmiņa: 330 MB
Nosaukums: msedge; ID: 23628; Atmiņa: 385.9 MB
Nosaukums: Discord; ID: 19836; Atmiņa: 393.5 MB
Nosaukums: explorer; ID: 7696; Atmiņa: 505.8 MB
Nosaukums: msedge; ID: 26300; Atmiņa: 522.8 MB
Nosaukums: msedge; ID: 25812; Atmiņa: 559.3 MB
Nosaukums: msedge; ID: 8728; Atmiņa: 923.4 MB
Nosaukums: Memory Compression; ID: 3304; Atmiņa: 960.8 MB
PS C:\Users\MX971B>
```

03

```
PS C:\Users\MX971B> # Atskaites ID: RVT-4417
PS C:\Users\MX971B>
PS C:\Users\MX971B> $servisi = Get-Service
PS C:\Users\MX971B> $darbojas = 0
PS C:\Users\MX971B> $aptureti = 0
PS C:\Users\MX971B>
PS C:\Users\MX971B> foreach ($serviss in $servisi) {
>>     if ($serviss.Status -eq "Running") {
>>         $darbojas++
>>     }
>>     elseif ($serviss.Status -eq "Stopped") {
>>         $aptureti++
>>     }
>> }
PS C:\Users\MX971B>
PS C:\Users\MX971B> $summa = $darbojas + $aptureti
PS C:\Users\MX971B>
PS C:\Users\MX971B> Write-Output "Darbojošos servisu skaits: $darbojas"
Darbojošos servisu skaits: 131
PS C:\Users\MX971B> Write-Output "Apturēto servisu skaits: $aptureti"
```

04 nav

05 nav

06

```
PS C:\Users\MX971B> # Atskaites ID: RVT-4417
PS C:\Users\MX971B>
PS C:\Users\MX971B> $draiveri = @(
>>     Get-CimInstance -ClassName Win32_PnPSignedDriver |
>>         Where-Object {
>>             $_.DriverProviderName -and
>>             $_.DriverProviderName -ne "Microsoft"
>>         } |
>>         Select-Object DeviceName, DriverProviderName, DriverVersion, InfName
>> )
PS C:\Users\MX971B>
PS C:\Users\MX971B> $draiveri |
>>     Export-Csv -Path ".\draiveri.csv" -NoTypeInformation -Encoding UTF8
PS C:\Users\MX971B>
PS C:\Users\MX971B> Write-Output "CSV ierakstu skaits: $($draiveri.Count)"
CSV ierakstu skaits: 35
PS C:\Users\MX971B> Write-Output "Fails saglabāts: draiveri.csv"
Fails saglabāts: draiveri.csv
PS C:\Users\MX971B>
```

07

```
PS C:\Users\MX971B> # Atskaites ID: RVT-4417
PS C:\Users\MX971B>
PS C:\Users\MX971B> $robeza = 10
PS C:\Users\MX971B>
PS C:\Users\MX971B> $diski = Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DriveType=3"
PS C:\Users\MX971B>
PS C:\Users\MX971B> foreach ($disks in $diski) {
>>     $izmers = [math]::Round($disks.Size / 1GB, 1)
>>     $brivaVieta = [math]::Round($disks.FreeSpace / 1GB, 1)
>>
>>     Write-Output "Disks: $($disks.DeviceID)"
>>     Write-Output "Kopējais izmērs: $izmers GB"
>>     Write-Output "Brīvā vieta: $brivaVieta GB"
>>
>>     if ($brivaVieta -lt $robeza) {
>>         Write-Output "BRĪDINĀJUMS: mazāk par $robeza GB brīvas vietas!"
>>     }
>>
>>     Write-Output ""
>> }
Disks: C:
Kopējais izmērs: 930.5 GB
Brīvā vieta: 85.4 GB
```

08 nav

09

```
PS C:\Users\MX971B> # Atskaites ID: RVT-4417
PS C:\Users\MX971B>
PS C:\Users\MX971B> $procesi = Get-CimInstance -ClassName Win32_Process
PS C:\Users\MX971B>
PS C:\Users\MX971B> foreach ($process in $procesi) {
>>     Write-Output "Process: $($process.Name); ID: $($process.ProcessId)"
>>
>>     $lietotajs = Invoke-CimMethod -InputObject $process -MethodName GetOwner -ErrorAction Continue
>>
>>     if ($null -ne $lietotajs -and $lietotajs.ReturnValue -eq 0) {
>>         Write-Output "Lietotājs: $($lietotajs.Domain)\$($lietotajs.User)"
>>     }
>>     else {
>>         Write-Output "Lietotāju noteikt neizdevās."
>>     }
>>
>>     Write-Output ""
>> }
Process: System Idle Process; ID: 0
Lietotāju noteikt neizdevās.

Process: System; ID: 4
Lietotāju noteikt neizdevās.

Process: Secure System; ID: 188
Lietotājs: NT AUTHORITY\SYSTEM

Process: Registry; ID: 232
Lietotājs: NT AUTHORITY\SYSTEM

Process: smss.exe; ID: 836
Lietotājs: NT AUTHORITY\SYSTEM

Process: csrss.exe; ID: 1268
Lietotājs: NT AUTHORITY\SYSTEM

Process: wininit.exe; ID: 1532
Lietotājs: NT AUTHORITY\SYSTEM

Process: csrss.exe; ID: 1540
Lietotājs: NT AUTHORITY\SYSTEM

Process: winlogon.exe; ID: 1632
Lietotājs: NT AUTHORITY\SYSTEM
```

10

```
PS C:\Users\MX971B> # Atskaites ID: RVT-4417
PS C:\Users\MX971B>
PS C:\Users\MX971B> $procesi = Get-Process
PS C:\Users\MX971B> Write-Output "Darbojošos procesu skaits: $($procesi.Count)"
Darbojošos procesu skaits: 316
PS C:\Users\MX971B>
PS C:\Users\MX971B> $servisi = Get-Service
PS C:\Users\MX971B> $darbojas = 0
PS C:\Users\MX971B> $aptureti = 0
PS C:\Users\MX971B>
PS C:\Users\MX971B> foreach ($serviss in $servisi) {
>>     if ($serviss.Status -eq "Running") {
>>         $darbojas++
>>     }
>>     elseif ($serviss.Status -eq "Stopped") {
>>         $aptureti++
>>     }
>> }
PS C:\Users\MX971B>
PS C:\Users\MX971B> Write-Output "Darbojošos servisu skaits: $darbojas"
Darbojošos servisu skaits: 130
PS C:\Users\MX971B> Write-Output "Apturēto servisu skaits: $aptureti"
Apturēto servisu skaits: 187
PS C:\Users\MX971B>
PS C:\Users\MX971B> $sakums = (Get-Date).AddHours(-48)
PS C:\Users\MX971B> $kludas = @(
>>     Get-WinEvent -FilterHashtable @{
>>         LogName = "System"
>>         Level = 2
>>         StartTime = $sakums
>>     } -ErrorAction SilentlyContinue
>> )
PS C:\Users\MX971B>
PS C:\Users\MX971B> Write-Output "Kļūdu skaits pēdējās 48 stundās: $($kludas.Count)"
Kļūdu skaits pēdējās 48 stundās: 15
PS C:\Users\MX971B>
PS C:\Users\MX971B> $diski = Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DriveType=3"
PS C:\Users\MX971B>
PS C:\Users\MX971B> foreach ($disks in $diski) {
>>     $brivaVieta = [math]::Round($disks.FreeSpace / 1GB, 1)
>>     Write-Output "Disks $($disks.DeviceID): brīvā vieta $brivaVieta GB"
>> }
Disks C:: brīvā vieta 85.4 GB
PS C:\Users\MX971B>
```

11

```
PS C:\Users\MX971B> # Atskaites ID: RVT-4417
PS C:\Users\MX971B>
PS C:\Users\MX971B> $adapteri = Get-NetAdapter
PS C:\Users\MX971B>
PS C:\Users\MX971B> $rezultats = foreach ($adapteris in $adapteri) {
>>     [PSCustomObject]@{
>>         Nosaukums = $adapteris.Name
>>         Statuss = $adapteris.Status
>>         MAC_adrese = $adapteris.MacAddress
>>         Atrums = $adapteris.LinkSpeed
>>     }
>> }
PS C:\Users\MX971B>
PS C:\Users\MX971B> $rezultats | Format-Table -AutoSize

Nosaukums                     Statuss      MAC_adrese        Atrums
---------                     -------      ----------        ------
Ethernet 2                    Disconnected 00-FF-F1-32-67-05 10 Mbps
Bluetooth Network Connection 2 Disconnected CC-28-AA-68-C4-A0 3 Mbps
Radmin VPN                    Up           02-50-C2-FE-A7-9C 100 Mbps
Ethernet                      Up           10-FF-E0-8C-66-0A 100 Mbps


PS C:\Users\MX971B>
PS C:\Users\MX971B> $rezultats | Export-Csv -Path ".\adapteri.csv" -NoTypeInformation -Encoding UTF8
PS C:\Users\MX971B>
```

Skaidrojums

01. Sistēmas informācijas iegūšana

Šajā uzdevumā izmantoju PowerShell komandas, lai iegūtu informāciju par datoru, operētājsistēmu, tās būvējuma numuru un PowerShell versiju. Skripts arī nosaka, cik procesi darbojas datorā. Tas palīdz iepazīt sistēmu un iegūt tās pamatinformāciju.

02. Procesu sakārtošana

Šajā uzdevumā ieguvu informāciju par datorā darbojošajiem procesiem un to izmantoto operatīvo atmiņu. Atlasīju desmit procesus ar lielāko atmiņas patēriņu un sakārtoju tos augošā secībā. Izmantoju arī Get-Member, lai apskatītu procesu objektu pieejamās īpašības un metodes.

03. Servisu skaitīšana

Šajā uzdevumā ieguvu informāciju par Windows servisiem un to statusiem. Ar foreach un if palīdzību atsevišķi saskaitīju darbojošos un apturētos servisus, kā arī noteicu kopējo servisu skaitu. Tas ļauj saprast, kā izmantot ciklus un nosacījumus datu apstrādei.

04. Automātisko servisu palaišana

Šajā uzdevumā meklēju automātiskās palaišanas servisus, kuri pašlaik ir apturēti. Vispirms izmantoju WhatIf režīmu, lai pārbaudītu, kādas darbības tiktu veiktas, faktiski nemainot servisu stāvokli. Pēc tam mēģināju palaist atrastos servisus. Dažu servisu palaišanai var būt nepieciešamas administratora tiesības.

05. Sistēmas kļūdu žurnāls

Šajā uzdevumā pārbaudīju Windows System notikumu žurnālu par pēdējām 48 stundām. Atlasīju kļūdu līmeņa notikumus, saskaitīju tos un noteicu, kurš notikumu

avots sastopams visbiežāk. Šādi var iegūt informāciju par iespējamām sistēmas problēmām.

06. Draiveru informācijas saglabāšana

Šajā uzdevumā ieguvu informāciju par instalētajiem ierīču draiveriem un atlasīju tos, kuru piegādātājs nav Microsoft. Izvēlētos datus saglabāju CSV failā, izmantojot Export-Csv. Šo failu var atvērt un apskatīt arī programmā Excel.

07. Diska brīvās vietas pārbaude

Šajā uzdevumā pārbaudīju datorā esošo lokālo disku izmēru un brīvo vietu. Skripts aprēķina informāciju gigabaitos un brīdina, ja brīvās vietas ir mazāk par 10 GB. Tas palīdz laikus pamanīt, ka diskā var pietrūkt vietas.

08. Divu komandu izpildes laika salīdzināšana

Šajā uzdevumā salīdzināju divus veidus, kā iegūt informāciju par Spooler servisu. Izmantoju Get-Service un Get-CimInstance, bet izpildes laiku mērīju ar Measure-Command. Rezultāti palīdz salīdzināt abu metožu izpildes laiku, ņemot vērā, ka tas var mainīties atkarībā no sistēmas noslodzes.

09. Procesu īpašnieku noteikšana

Šajā uzdevumā mēģināju noteikt, kurš lietotājs ir palaidis katru procesu. Izmantoju Get-CimInstance un Invoke-CimMethod ar GetOwner metodi. Dažiem procesiem lietotāja informācija var nebūt pieejama piekļuves tiesību vai sistēmas ierobežojumu dēļ. Šajā uzdevumā svarīgi ir arī pareizi apstrādāt kļūdas.

10. Sistēmas atkārtota pārbaude

Šajā uzdevumā atkārtoti pārbaudīju procesu un servisu skaitu, kļūdu ierakstus pēdējo 48 stundu laikā un diska brīvo vietu. Iegūtos rezultātus salīdzināju ar iepriekšējo uzdevumu datiem. Rezultāti var atšķirties, jo sistēmas darbības laikā mainās procesu un servisu stāvokļi, kā arī žurnālos tiek reģistrēti jauni notikumi.

11. Tīkla adapteru informācija

Šajā uzdevumā ieguvu informāciju par datorā pieejamajiem tīkla adapteriem. Apskatīju to nosaukumus, statusus, MAC adreses un savienojuma ātrumu. Rezultātus parādīju tabulā un saglabāju CSV failā. Šī informācija palīdz pārbaudīt datora tīkla savienojumus un adapteru stāvokli.